

การเปรียบเทียบ การสร้างชุดทดสอบโดยปัญญาประดิษฐ์ กับอัลกอริทึมสร้างชุดทดสอบอัตโนมัติ

AI-Assisted Testing vs. Automatic Test Case Generation Algorithms: A Benchmark and Test Coverage Evaluation

กลุ่มที่ 7

Presented By
นางสาวพิไลพร คำเวียง
นางสาวหทัยพัชร วิสุทธีธรรม



ความเป็นมาและความสำคัญของปัญหา

ปัญหา

ซอฟต์แวร์มีบทบาทสำคัญต่อระบบสารสนเทศและธุรกิจ การทดสอบจึงเป็นกระบวนการสำคัญในการประกันคุณภาพ แต่การสร้างกรณีทดสอบด้วยตนเองใช้เวลานาน โดยเฉพาะระบบที่มีความซับซ้อนสูง

จึงทำการทดลองเพื่อศึกษาและเปรียบเทียบประสิทธิภาพของการสร้างชุดทดสอบอัตโนมัติระหว่างแนวทางที่ใช้อัลกอริทึมกับแนวทางที่ใช้ปัญญาประดิษฐ์



วัตถุประสงค์และขอบเขตของโครงการ

วัตถุประสงค์

- ศึกษาและทดลองใช้เครื่องมือสร้าง Test Case โดยอัตโนมัติด้วยแนวทางที่แตกต่างกัน
- เปรียบเทียบประสิทธิภาพของเครื่องมือในการสร้าง Test Case ทั้งด้าน Coverage และต้นทุนการประมวลผล

ขอบเขต

- ใช้ชุดข้อมูล Defects4J จำนวน 17 โปรเจกต์ รวม 854 ข้อบกพร่อง
- แนวทาง Search-based: Grey Wolf Optimizer (GWO) ร่วมกับ EvoSuite
- แนวทาง Coverage-guided Fuzzing: Zest / JQF
- ปัญญาประดิษฐ์: Gemini-3.7-flash และ DeepSeek-v4-flash

แนวทางการสร้างชุดทดสอบ

GWO

Grey Wolf Optimizer ประยุกต์ใช้ร่วมกับ EvoSuite แนวทาง Search-based โดยจำลองพฤติกรรมฝูงหมาป่าเพื่อค้นหาชุดทดสอบที่เหมาะสม

Zest

Coverage-Guided Fuzzing ผ่าน JQF ใช้ Code Coverage เป็นข้อมูลย้อนกลับในการคัดเลือก input ที่มีประโยชน์

Gemini-3.7-flash

โมเดลภาษาขนาดใหญ่ วิเคราะห์โครงสร้างและพฤติกรรมของ Source Code แล้วสร้าง Test Case โดยตรง

DeepSeek-v4-flash

โมเดลภาษาขนาดใหญ่อีกตัว เรียกผ่าน KKU IntelSphere API พร้อมกลไกปรับปรุง Prompt แบบวนซ้ำ



ชุดข้อมูล DEFECTS4J

17

โปรเจค

854

Bug

Defects4J เป็นชุดข้อมูล Benchmark สำหรับงานวิจัยด้านการทดสอบซอฟต์แวร์ รวบรวมโครงการโอเพนซอร์สภาษา Java ที่มีข้อบกพร่องจริง และเวอร์ชันก่อน/หลังการแก้ไข

Codec

Collections

Compress

Csv

Gson

JacksonXml

Jsoup

JXPath

Lang

Math

ตัวชี้วัดที่ใช้ประเมินผล

Line Coverage

สัดส่วนบรรทัดคำสั่งที่ถูกเรียกใช้งานระหว่างการทดสอบ

Branch Coverage

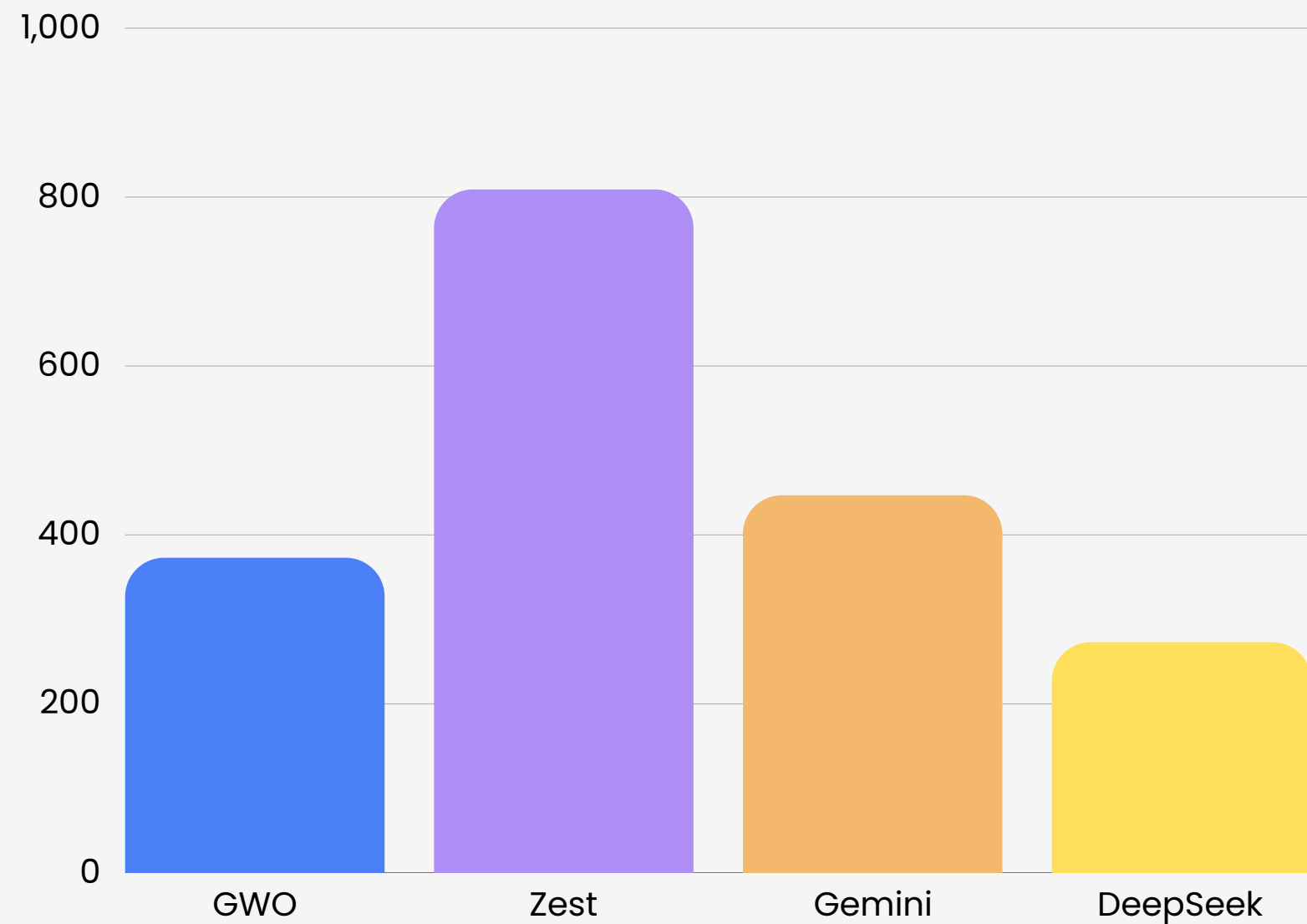
สัดส่วนกิ่งการตัดสินใจ (if, switch, while, for) ที่ถูกทดสอบให้ทำงานทั้งจริงและเท็จ

Condition Coverage

สัดส่วนเงื่อนไขย่อยภายในนิพจน์ตัดสินใจที่ถูกทดสอบให้เป็นทั้งจริงและเท็จ

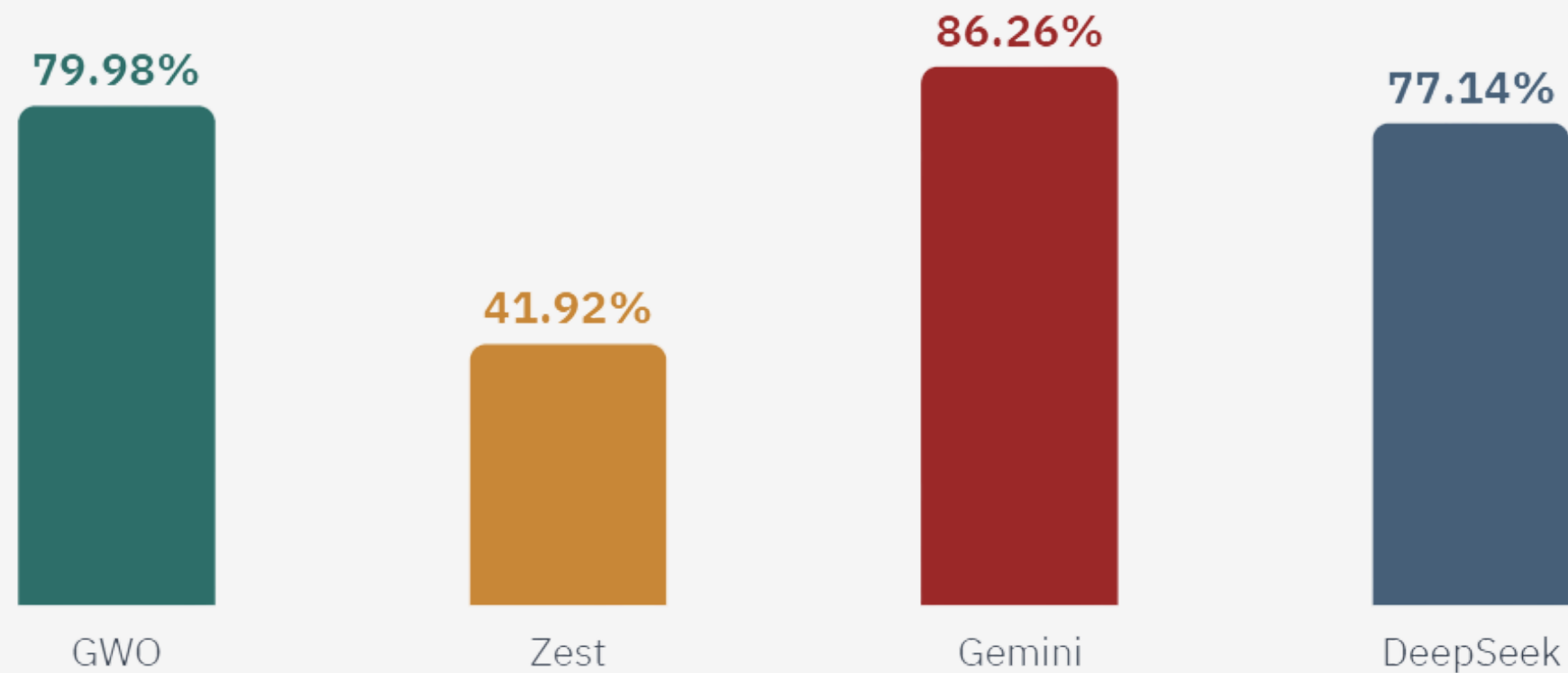


ภาพรวมจำนวน BUG ที่ทดสอบได้ในแต่ละวิธี



จากทั้งหมด 854 Bug ใน 17 โปรเจค แต่ละวิธีสามารถดำเนินการทดลองและได้ผลลัพธ์ไม่เท่ากัน เนื่องจากข้อจำกัดในการสร้าง หรือวัด Coverage จึงใช้ Common Bugs ที่ทุกวิธีมีข้อมูลร่วมกันในการเปรียบเทียบขึ้นถัดไป

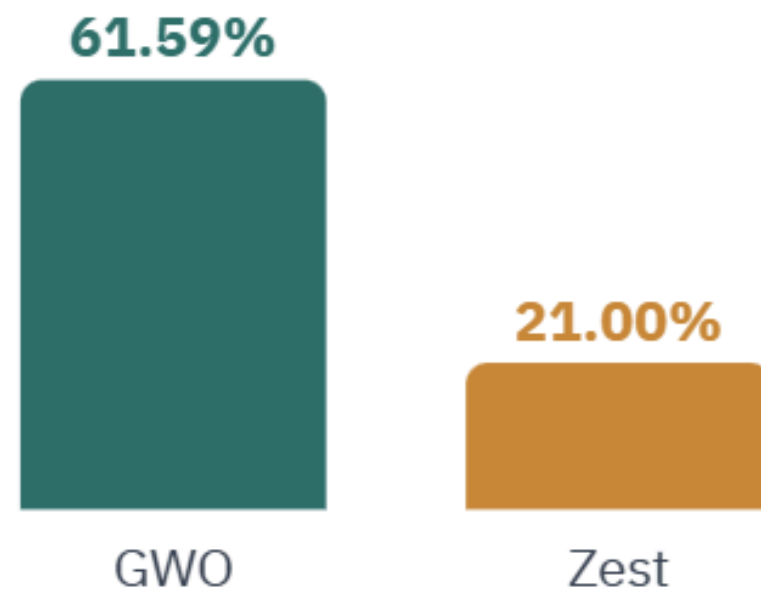
LINE COVERAGE โดยรวม (COMMON BUGS 249 บั๊ก)



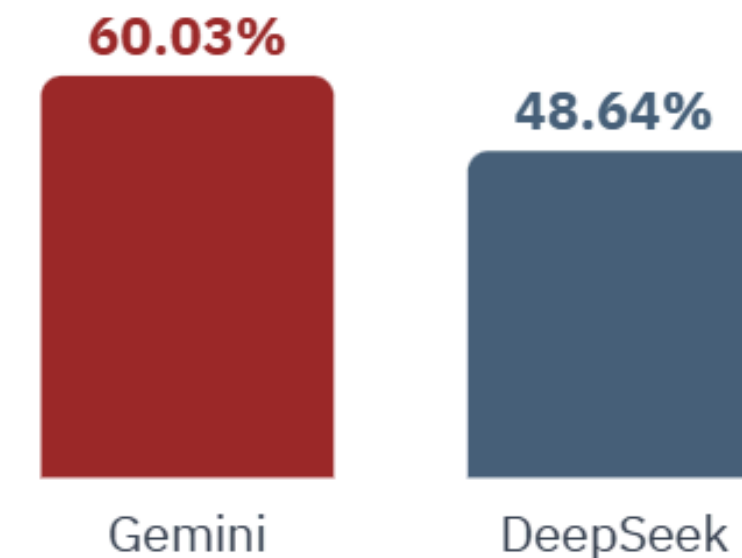
Gemini-3.7-flash ให้ Line Coverage สูงสุด 86.26% สูงกว่า GWO รว 6.3 จุด และ สูงกว่า DeepSeek รว 9.1 จุด Zest ต่ำกว่าวิธีอื่นอย่างชัดเจน ต่ำกว่า Gemini ถึง 44.3 จุดเปอร์เซ็นต์

BRANCH และ CONDITION COVERAGE

Branch Coverage · GWO vs Zest (Common Bugs 689)



Condition Coverage · Gemini vs DeepSeek (Common Bugs 288)



GWO และ Zest เปรียบเทียบกันได้เฉพาะ Branch Coverage ส่วน Gemini และ DeepSeek เปรียบเทียบกันได้เฉพาะ Condition Coverage เนื่องจากรูปแบบข้อมูลที่แต่ละเครื่องมือบันทึกต่างกัน

TEST SUITE CHARACTERISTICS

Grey Wolf Optimizer

- Test Cases สูงสุด: Lang 154.68 · ต่ำสุด: Mockito 8.72
- Suite Length สูงสุด: JacksonCore 535.62

Zest

- JUnit Tests สูงสุด: Codec 101.94 · ต่ำสุด: JacksonXml 12.50
- Fuzz Time ส่วนใหญ่ 120–128 วินาที

Gemini-3.7-flash

- Total Tokens สูงสุด: Closure 5,552,488 (162 Bugs)
- Average Tokens สูงสุด: JacksonCore 29,601

DeepSeek-v4-flash

- Total Tokens สูงสุด: Jsoup 4,365,164
- Average Tokens สูงสุด: JacksonCore 46,510

สาเหตุความล้มเหลวของชุดทดสอบจาก AI

อ้างอิง Class/Package ที่ไม่มีอยู่จริง

แบบจำลองจินตนาการ Class ที่ไม่มีอยู่จริงในโครงสร้างโปรเจกต์ (Hallucination)

จำนวน/ชนิด Argument ผิดพลาด

เรียก Constructor หรือ Method ด้วยจำนวน Argument ไม่ตรงกับ Signature จริง

คาดเดาชนิดค่าที่คืนกลับผิดพลาด

สมมติ Return Type ผิดจากที่ Method คืนค่าจริง เช่น Array แทนที่จะเป็น List<T>

Override Method ไม่ครบใน Anonymous/Inner Class

พบมากในโครงการที่มี Interface ซ้อนกันหลายชั้น เช่น Mockito

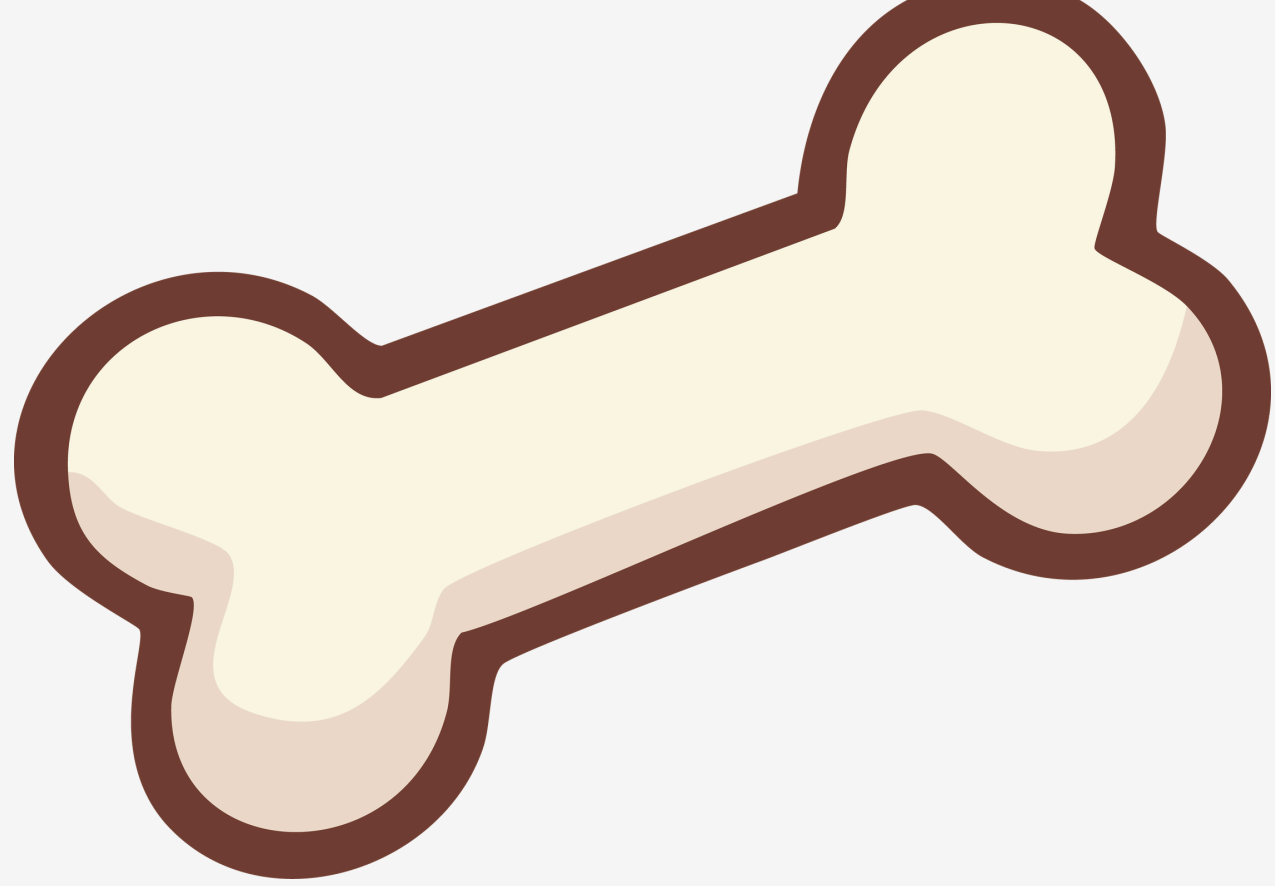
FAIL

အနုပညာ



THANK YOU





สมาชิก

นางสาวพิไลพร คำเวียง 673380286-0 sec.1

นางสาวหทัยพัทธ วิสุทธีธรรม 673380297-5 sec.1

